Table of Contents

Table of Contents	1	Fraction Class	21
Simple Graph Stuff	2-5	SCC's	22
Binary Search	6	Ford Fulkerson	23
2D Grids (PSum, kadanes)	7	String Stuff 1	24-26
Math Stuff	8-10	TSP and Permutations	27
Segment Trees	11-14	2D Binary Index Tree	28
Matrix Expo	15	Palindrome Tree	29
SQRT Blocking Template	16	Formula/Bitwise Stuff	30
Tree Misc	17	DSU++	31
LCA's, Min on path	18-20	Dynamic Connectivity	32

Template

```
typedef long long ll;
typedef long double ld;
typedef pair<int, int> pii;
typedef vector<int> vi;
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
template<class T>
ostream & operator << (ostream & os, vector<T> v) {
   for (auto x : v) os << x << ' ';
   os << "\n"; return os;
}
```

Bash

```
cd ~
code .bashrc
alias cpp="g++ -std=c++17 -Wall -Wshadow -Wconversion "
alias r="./a.out"
export PATH="$HOME/bin:$PATH"
```

Hashing: 5246ca (this hackpack)

```
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
Run: chmod +x hash.sh, in terminal
./hash.sh < filename
```

Hashing Luke: (**NOT THIS HACKPACK**) d41d8c

```
sed -r '/(assert|include|pragma)/d' | cpp -fpreprocessed -P | tr -d
'[:space:]' | md5sum | cut -c-6
```

Binary Search

11111111111111111111111111111111	00000000000000000000000000000000

`while(lo!=hi){`
`  mid = lo+(hi-lo+1)/2;`
`  if(good(mid)) lo = mid;`
`  else hi = mid-1;`
`}`
`//answer in lo/hi (not mid)`
`//finds last 1`
	`while(lo!=hi){`
	`  mid = lo+(hi-lo)/2;`
	`  if(good(mid)) lo = mid+1;`
	`  else hi = mid;`
	`}`
	`//answer in lo/hi (not mid)`
	`//finds first 0`

Ternary Search Doubles

```
ld ternary_search(ld l, ld r){
   for(int i = 0; i < 200; i++){
      ld m1 = l+(r-l)/3.0;
      ld m2 = r-(r-l)/3.0;
      if(f(m1) >= f(m2)) l = m1;
      else r = m2;
   }
   return func(l); //for min
}
```

Integer Ternary Search

```
while(B - A > 4){
    int m1 = (A + B) / 2;
    int m2 = (A + B) / 2 + 1;
    if(f(m1) > f(m2)) A = m1;
    else B = m2;
}

ans = inf;
for(int i = A; i <= B; i++)
    ans = min(ans , f(i));
```

Disjoint Set Union: 75e9df

```
struct dsu{
   vector<int> par,siz;
   dsu(int n){
       siz = par = vector<int>(n,1);
       iota(par.begin(),par.end(),0);
   }
   int find(int a){
       if(par[a] != a) par[a] = find(par[a]);
       return par[a];
   }
   bool join(int a, int b){
       a = find(a); b = find(b);
       if(a == b) return false;
       if(siz[a] < siz[b]) swap(a,b);
       par[b] = a; siz[a]+=siz[b];
       return true;
   }
};
```

MST (Kruskals)

```
ll total = 0; //not always int!
sort(edges.begin(), edges.end());
for (auto [w,u,v] : edges) {
  if (!ds.join(u,v)) continue;
  total += w;
}
```

Dijkstra's, graph is vector<vector<pair<to,weight>>>

```
typedef pair<ll,int> pli;
ll INF = 1e15;
priority_queue<pli,vector<pli>,greater<pli>> pq;
vector<ll> dist(n,INF);
vector<int> par(n,-1); //path buildback
dist[source] = 0;
pq.emplace(0,source);
while(!pq.empty()){
   auto[cost,u] = pq.top(); pq.pop();
   if(cost > dist[u]) continue;
   for(auto[v, w]: graph[u]){
       if(cost+w < dist[v]){
           dist[v] = cost+w;
           pq.emplace(cost+w,v);
           par[v] = u; //path buildback
       }
   }
}
vector<int> path;
for(int at = sink; at != -1; at = par[at])
   path.push_back(at);
reverse(path.begin(),path.end());
```

Floyd Worshalls, (All pairs Shortest Path) n =, graph = adjMatrix, O(n^3)

```
ll INF = 1e17; //DONT FORGET TO ZERO THE DIAGONAL, everything else inf
for(int p = 0; p < n; p++)
   for(int u = 0; u < n; u++)
       for(int v = 0; v < n; v++)
           if(graph[u][p] < INF && graph[p][v] < INF)
               graph[u][v] = min(graph[u][v],graph[u][p]+graph[p][v]);
//find negative cycles, ignore if no negative edgeweights
for(int i = 0; i < n; i++)
   for(int j = 0; j < n; j++)
       for(int t = 0; t < n; t++)
          if(graph[i][t] < INF && graph[t][t] < 0 && graph[t][j] < INF)
               graph[i][j] = -INF;
```

Bellman Ford SSSP with negative Edge Weights,(n = # nodes), O(V*E)

```
ll INF = 1e16; vector<int> path;
vector<tuple<int,int,ll>> edgelist; //u,v,w
vector<ll> dists(n,INF); //can be changed to n,0 to find all cycles
vector<int> par(n,-1); vector<bool> nCycle(n);
dists[0] = 0; int cycleStart;
for(int i = 0; i < n; i++){
    cycleStart = -1;
    for(auto [u,v,w]: edgelist){
        if(dists[u] >= INF) continue;
        if(dists[u] + w < dists[v]){
            dists[v] = max(-INF,dists[u]+w);
            par[v] = u; cycleStart = v;
            if(i+1 == n) nCycle[v] = true;
        }
    }
} //if cycleStart == -1 no negative cycle
for(int i = 0; i < n; i++) cycleStart = par[cycleStart];
for(int cur = cycleStart; true; cur = par[cur]){
    path.push_back(cur); if(cur == cycleStart && path.size()>1) break;
} //don't forget this bracket
reverse(path.begin(),path.end());
//for ssp from u -> v, dfs for reachable nodes of u, reachable nodes
//on rev edge graph for v see if any cycle node is reachable from both
//to see if path is -inf
```

Topological Sort

```
queue<int> q; vector<int> ordering; vector<int> indegree(n,0);
//priority_queue<int, vector<int>, greater<int>> q; //for lexic minimal
for(auto u: graph) for(auto v: u) indegree[v]++;
for(int i = 0; i < n; i++) if(!indegree[i]) q.push(i);
while(!q.empty()){
   int u = q.front(); q.pop();
   ordering.push_back(u);
   for(auto v: graph[u]){
       if(--indegree[v] == 0) q.push(v);
   }
} //if ordering.size() != # nodes no valid ordering
```

BFS (source = startpoint, sink = endpoint, n = #nodes, O(V + E)):

```
vector<int> dist(n,-1);
vector<int> par(n,-1);
queue<int> q; q.push(source);
dist[source] = 0;
while(!q.empty()){
   int u = q.front(); q.pop();
   for(auto v: graph[u]){
       if(dist[v] != -1) continue;
       dist[v] = dist[u] + 1;
       par[v] = u; //Path Build back
       q.push(v);
   }
}
//Path build back, if path[0] != source NO PATH
vector<int> path;
for(int at = sink; at != -1; at = par[at]) path.push_back(at);
reverse(path.begin(),path.end());
```

DFS cycle Detection, call on vis[u] = 0 not just one start point.

```
bool findcycle(int u){
    vis[u] = 1;
    for(auto v: graph[u]){
        if(!vis[v]){ par[v] = u; if(findcycle(v)) return true;
        } else if(vis[v] == 1){
            cStart = v; cEnd = u; //global variables
            return true;
        }
    }
    vis[u] = 2;
    return false;
}
//Recover Cycle, some other code not included
vector<int> path;
path.push_back(cStart);
for(int cur = cEnd; cur != cStart; cur = par[cur]) path.push_back(cur);
path.push_back(cStart);
reverse(path.begin(),path.end());
```

2D Prefix Sum: 91a17f

```
void pref(vector<vector<int>> &grid){ //CHANGE TO LONG IF NEED!
    for(auto &row: grid) row.push_back(0);
    grid.emplace_back(grid[0].size());
    for(int i = grid.size()-2; i >= 0; i--){
        for(int j = grid[0].size()-2; j>=0; j--){
            grid[i][j]+=(grid[i+1][j]+grid[i][j+1]-grid[i+1][j+1]);
        }
    }
}
//upper left, bottom right, CHANGE TO LONG IF NEED
int query(int up, int le, int bo, int ri,vector<vector<int>> &grid){
    return grid[up][le]-grid[bo+1][le]-grid[up][ri+1]+grid[bo+1][ri+1];
}
```

Max Sub Rectangle (N^3) (assumes 0 size rect allowed): e96749

```
ll kadanes2D(vector<vector<ll>> &grid){
    ll INF = 1e16; ll result = -INF;
    for(auto &row: grid) partial_sum(row.begin(),row.end(),row.begin());
    for(int l = 0; l < grid.size(); l++){
        for(int r = l+1; r < grid.size(); r++){
            ll curmax = -INF; ll sum = 0;
            for(int i = 0; i < grid[0].size(); i++){
                sum += (grid[i][r]-grid[i][l]);
                curmax = max(sum,curmax); sum = max(sum,0ll);
            }
            result = max(result,curmax);
        }
    }
    return result;
}
```

Rotation Trick

```
vector<vector<int>> rotated(2*n-1, vector<int>(2*n-1));
for(int i = 0; i < n; i++)
   for(int j = 0; j < n; j++)
      rotated[i + j][j - i + n - 1] = grid[i][j];
```

ModPow: 08bf0d

```
ll modpow(ll b, ll p){
   ll ans = 1;
   while(p){
       if(p&1) ans = (ans*b) % mod;
       b = (b*b)%mod;
       p>>=1;
   }
   return ans;
}
```

Mod Inverse

$$\frac{1}{x} \equiv x^{p-2} \; mod\, p, \; where\; p\; is\; prime$$

Choose: 04ccb3

```
vector<ll> facs(maxn+1);
vector<ll> inv(maxn+1);
facs[0] = 1;
for(int i = 1; i <= maxn; i++) facs[i] = (facs[i-1]*i)%mod;
inv[maxn] = fastexpo(facs[maxn],mod-2); //mod inverse
for(int i = maxn; i >= 1; i--){
   inv[i-1] = (inv[i]*i)%mod;
}

ll choose(ll n, ll k, vector<ll> &facs, vector<ll> &inv){
   ll top = facs[n];
   ll bottom = (inv[k]*(inv[n-k]))%mod;
   return (top*bottom) % mod;
}

//stars and bars s = stars, b = bars choose(s+b-1,b-1)
// Catalan numbers  (2n)! / (n+1)!n!
// Computes number of valid brackets sequences with 2N brackets
```

Prime Sieve: 525316

```
vector<bool> sieve(int n){
   vector<bool> ip(n+1,1);
   ip[0] = ip[1] = 0;
   for(int i = 2; i*i <= n; i++){
       if(ip[i]){
           for(int j = i*i; j<=n; j+=i) ip[j] = 0;
       }
   }
  return ip; //or return primes
}
```

Factorize: 6169d5

```
vector<ll> factorization(ll n){
   vector<ll> facs; //primes is global compute with sieve
   for(ll f : primes){
       if(f*f>n) break;
       while(n%f == 0){
           facs.push_back(f);
           n /= f;
       }
   }
   if(n>1) facs.push_back(n);
   return facs;
}
```

Euler Totient(Number of integers 1-N Coprime to N): 8605f5

```
ll phi(ll n){
   ll ans = n;
   for(ll i = 2; i*i <= n; i++){
       bool f = 0;
       while(n % i == 0)
           f = (n /= i);
       if(f){ans-=(ans/i);}
   }
   if(n>1){ans-=ans/n;}
   return ans;
}
```

Lexicographically Kth Permutation, O(n^2): 6a818d

```
vector<int> kthperm(int n, ll k){
   //0 indexed (k = 0 returns sorted array) //k > n! > 20! is bad
   vector<int> res; vector<int> order(n);
   vector<ll> fact(n+1,1);
   iota(order.begin(),order.end(),1);
   for(ll i = 2; i <=n; i++)
       fact[i] = fact[i-1] * i;
   for(int i = n-1; i>=0; i--){
       ll temp = k/fact[i];
       k%= fact[i];
       res.push_back(order[temp]);
       order.erase(order.begin()+temp);
   }
   return res;
}
```

Factorize O(log(n)) O(N) precomp: 415292

```
vector<int> facprecomp(int n){
   vector<int> f(n+1,-1);
   for(int i = 2; i<=n; i++){
       if(f[i] != -1) continue;
       for(int j = i; j <=n; j+=i)
           f[j] = i;
   }
   return f;
}

vector<int> factorize(int n, vector<int> &fa){
   vector<int> res;
   while(fa[n] != -1){
       res.push_back(fa[n]);
       n/=fa[n];
   }
   return res;
}
```

Segment Tree Basic: 34a43e

```
struct node{
   int lo,hi,mid,val;
   node *left; node *right;
   node(int l, int r){
       lo = l; hi = r;
       mid = (lo+hi)/2;
       val = IDEMPOTENT_VALUE;
       if(l==r) return;
       left = new node(l,mid);
       right = new node(mid+1,r);
   }
   int query(int l, int r){
       if(hi<l||r<lo) return IDEMPOTENT_VALUE;
       if(l<=lo&&hi<=r) return val;
       return merge(left->query(l,r),right->query(l,r));
   }
   void update(int i){
       if(lo==hi) return void(val++);
       if(i<=mid) left->update(i);
       else right->update(i);
       val = merge(left->val,right->val);
   }
};
```

Segtree Walk with start index example //first el >= k as example

```
int walk(int k, int le){
   if(val<k || hi < le ) return -1; //fails requirements
   if(lo==hi) return lo; //single node
   int ans = left->walk(k,le);
   if(ans == -1) ans = right->walk(k,le);
   return ans;
}
```

Segment Tree Implicit: b9f05a

```
struct ST{
   ll lo,hi,mid;
   ll val;
   ST *left = nullptr; ST *right = nullptr;
   ST(int l, int r){
       lo = l; hi = r;
       mid = (lo+hi)/2;
       val = 0;
   }
   void check(){
       if(!left){
           left = new ST(lo,mid);
           right = new ST(mid+1,hi);
       }
   }
   ll query(ll l, ll r){
       if(r < lo || l > hi) return //DEFAULT_VALUE;
       if(l <= lo && hi <= r) return //val;
       check();
       return merge(left->query(l,r),right->query(l,r))
   }
   void update(ll i, int v){
       if(lo == hi) return //void("change val with v");
       check();
       if(i<=mid) left->update(i,v);
       else right->update(i,v);
       val = merge(left->val,right->val);
   }
};
```

Segment Tree Lazy: 7e6fc7

```
struct node{
    int lo, hi, mid;
    ll val,delta;
    node *left; node *right;
    node(int l, int r){
        lo = l; hi = r; mid = (lo+hi)/2;
        val = delta = 0;
        if(lo==hi) return;
        left = new node(l,mid);
        right = new node(mid+1,r);
    }

    void prop(){
        left->delta +=delta;
        right->delta +=delta;
        delta = 0;
    }

    ll value(){ return val+(delta*(hi-lo+1)); }
    ll merge(ll left, ll right){ return left+right; }
    ll query(int l, int r){
        if(l>hi||r<lo) return 0;
        if(l<=lo&&hi<=r) return value();
        prop();
        val = merge(left->value(),right->value());
        return left->query(l,r)+right->query(l,r);
    }

    void update(int l, int r, int v){
        if(l>hi||r<lo) return;
        if(l<=lo&&hi<=r) return void(delta+=v);
        prop();
        left->update(l,r,v); right->update(l,r,v);
        val = merge(left->value(),right->value());
    }
};
```

Persistent Segment Tree, *Point Set*, Range Sum: aed623

```
struct node{
   int lo,hi,mid;
   ll val;
   node *left; node *right;
   node(int l, int r){
       lo = l; hi = r;
       mid = (lo+hi)/2;
       val = vals[lo];
       if(l==r) return;
       left = new node(l,mid);
       right = new node(mid+1,r);
       val = left->val+right->val;
   }
   ll query(int l, int r){
       if(hi < l || r < lo) return 0;
       if(l <= lo && r >= hi) return val;
       return left->query(l,r)+right->query(l,r);
   }
   node(node *other){ *this = *other; }
   node* update(int i, ll x){
       node *res = new node(this);
       if(lo==hi){ res->val = x; return res;}
       if(i<=mid) res->left = left->update(i,x);
       else res->right = right->update(i,x);
       res->val = res->left->val + res->right->val;
       return res;
   }
};
```

Matrix Expo Class: 9e81af

```
const ll mod = ?
const int dim = ?
using Matrix = array<array<ll,dim>,dim>;
#define rep(i,a,b) for(int i = a; i < (b); i++)
//Arrays are not zero initialized
Matrix mul(Matrix a, Matrix b){
    Matrix res;
    rep(i,0,dim) rep(j,0,dim) res[i][j] = 0;
    rep(i,0,dim) rep(j,0,dim)
        rep(k,0,dim){
            res[i][j] += a[i][k]*b[k][j];
            res[i][j] %=mod;
        }
    return res;
}
//Arrays are not zero initialized
ll dot(const vector<ll> &b, Matrix m){
    ll ans = 0;
    rep(i,0,dim) ans = (ans+(m[0][i]*b[i])) % mod;
    return ans;
}
//Arrays are not zero initialized
Matrix expo(Matrix base, ll e){
    Matrix ans; rep(r,0,dim) rep(c,0,dim) ans[r][c] = (r==c);
    while(e){
        if(e&1) ans = mul(ans,base);
        base = mul(base,base);
        e/=2;
    }
    return ans;
}
//Arrays are not zero initialized
```

Sqrt Decomp (Blocking): 4d4bca

```
struct Sqrt{
    int INF = INT_MAX;
    const int block_size = 450; //sqrt 2e5 ~ 450
    vector<int> nums, blocks;
    Sqrt(vector<int> &arr){
        nums = arr;
        blocks = vector<int>((arr.size()/block_size)+1,INF);
        for(int i = 0; i < nums.size(); i++){
            blocks[i/block_size] = min(blocks[i/block_size],nums[i]);
        }
    }

    void update(int ind, int v){
        nums[ind] = v;
        int b_num = ind/block_size;
        blocks[b_num] = INF;
        for(int i = b_num*block_size; i < (b_num+1)*(block_size); i++){
            blocks[i/block_size] = min(blocks[i/block_size],nums[i]);
        }
    }

    int query(int l, int r){
        int ans = INF; //default starting value
        while(l<r && (l%block_size) && l) ans = min(ans,nums[l++]);
        while(l+block_size<=r) {
            ans = min(ans,blocks[l/block_size]);
            l += block_size;
        }
        while(l<=r) ans = min(ans,nums[l++]); //other partial
        return ans;
    }
};
```

Euler Tour: bc0739

```
vector<int> pre,post;
vector<vector<int>> graph;
int t = 0;
void dfs(int u, int p){
   pre[u] = t; t++; //increment here and post for path sum stuff
   for(int v: graph[u]) if(v!=p) dfs(v,u);
   post[u] = t; t++; //or no inc, post[u] = t-1, for just subtree
}
dfs(0,-1);
//query structure of size 2*n
//updates on pre[u], queries from pre[u] to post[u]
//path sum from node u to root by storing negative at post[u], pre[u]
//query pre[root] to (post[u] - 1)
```

Tree Max distance for all nodes (dfs1 then 2, (0,-1,graph)): 75f137

```
vector<int> maxL,smax,ch;
void dfs(int u, int p, vector<vector<int>> &graph){
 for(auto v: graph[u]){
       if(v==p) continue; dfs(v,u,graph);
       if(maxL[v]+1 > maxL[u]){
           smax[u] = maxL[u]; maxL[u] = maxL[v]+1; ch[u] = v;
       }else if(maxL[v]+1>smax[u])
           smax[u]=maxL[v]+1;
 }
}
void dfs2(int u, int p,vector<vector<int>> &graph){
   for(auto v: graph[u]){
       if(v==p) continue;
       if(ch[u] == v){
           if(maxL[v] < smax[u]+1){
               smax[v] = maxL[v]; maxL[v] = smax[u]+1; ch[v] = u;
           }else smax[v] = max(smax[v],smax[u]+1);
       }else
           smax[v]=maxL[v]; maxL[v]=maxL[u]+1; ch[v]=ch[u];
       dfs2(v,u,graph);
   }
}
```

LCA with Binary Lifting: 6d44f7

```
typedef vector<int> vi;
typedef vector<vi> vii;
struct bl{
   vii g, jmp;
   vi dep; int n;
   bl(vii &graph){
       g = graph; n = g.size(); dep = vi(n);
       jmp = vii( __lg(n)+1,vi(n));
       dfs(0,0,0);
   }
   void dfs(int u, int p, int d){
       jmp[0][u] = p; dep[u] = d;
       for(int i = 1; i < (int)jmp.size(); i++)
           jmp[i][u] = jmp[i-1][jmp[i-1][u]];
       for(auto v: g[u]) if(v != p) dfs(v,u,d+1);
   }
   int kth(int u, int k){
       for(int i = 0; i < (int)jmp.size(); i++)
           if(k & (1<<i)) u = jmp[i][u];
       return u;
   }
   int lca(int u, int v){
       if(dep[u] > dep[v]) swap(u,v); //v is now deeper
       v = kth(v,dep[v]-dep[u]); if(u==v) return u;
       for(int i = (int)jmp.size()-1; i>=0; i--){
           if(jmp[i][u] != jmp[i][v]){
               u = jmp[i][u];
               v = jmp[i][v];
           }
       }
       return jmp[0][u];
   }
};
```

Min On Path for Tree (No Updates): e383da

```
typedef vector<int> vi; typedef vector<vi> vii;
typedef vector<vector<pair<int,int>>> vpi;
struct bl{
   vpi g; vii jmp; vi dep; int n; vii mi;
   bl(vpi &graph){
       g = graph; n = g.size(); dep = vi(n);
       mi = jmp = vii( __lg(n)+1,vi(n)); dfs(0,0,0,0);
   }
   void dfs(int u, int p, int d,int ew){
       jmp[0][u] = p; dep[u] = d; mi[0][u] = ew;
       for(int i = 1; i < (int)jmp.size(); i++){
           jmp[i][u] = jmp[i-1][jmp[i-1][u]];
           mi[i][u] = min(mi[i-1][u],mi[i-1][jmp[i-1][u]]);
       }
       for(auto [v,w]: g[u]) if(v != p) dfs(v,u,d+1,w);
   }
   pair<int,int> kth(int u, int k){
       int res = INT_MAX;
       for(int i = 0; i < (int)jmp.size(); i++)
           if(k & (1<<i)){
               res = min(res,mi[i][u]); u = jmp[i][u];
           }
       return {u,res};
   }
   int smallest(int u, int v){
       int lc = lca(u,v);
       int ju = dep[u]-dep[lc]; int jv = dep[v]-dep[lc];
       return min(kth(u,ju).second,kth(v,jv).second);
   }
   int lca(int u, int v){
       if(dep[u] > dep[v]) swap(u,v);
       v = kth(v,dep[v]-dep[u]).first; if(u==v) return u;
       for(int i = (int)jmp.size()-1; i>=0; i--)
           if(jmp[i][u] != jmp[i][v])
               u = jmp[i][u], v = jmp[i][v];
       return jmp[0][u];
   }
};
```

O(1) LCA With RMQ: 8bc8a9

```
typedef vector<int> vi;
typedef pair<int,int> pii;
struct LCA{
   vi depth,pos; vector<pii> walk;
   void dfs(int u, int p, int d){
       depth[u] = d;
       pos[u]=walk.size();
       walk.emplace_back(d,u);
       for(auto v: adj[u]){
           if(v!=p){
               dfs(v,u,d+1);
               walk.emplace_back(d,u);
           }
       }
   }

   vector<vi> adj; RMQ<pii>*rmq;
   LCA(vector<vi> &g){
       adj=g; depth = pos = vi(g.size());
       dfs(0,-1,0);
       rmq = new RMQ<pii>(walk);
   }
   int lca(int u, int v){
       if(pos[u]>pos[v]) swap(u,v);
       return rmq->query(pos[u],pos[v]+1).second;
   }
   int dist(int u, int v) {return depth[u]+depth[v]-2*depth[lca(u,v)];}
};
```

Fraction Class (Seba's): c1af13

```
template<class T> struct Q {
   T a, b;
   Q(T p, T q = 1) {
       T g = gcd(p, q);
       a = p / g;
       b = q / g;
       if (b < 0) a = -a, b = -b;
   }
   T gcd(T x, T y) const { return __gcd(x, y); }
   Q operator+(const Q& o) const {
       return {a * o.b + o.a * b, b * o.b};
   }
   Q operator-(const Q& o) const {
       return *this + Q(-o.a, o.b);
   }
   Q operator*(const Q& o) const { return {a * o.a, b * o.b}; }
   Q operator/(const Q& o) const { return *this * Q(o.b, o.a); }
   Q recip() const { return {b, a}; }
   int signum() const { return (a > 0) - (a < 0); }
   bool operator<(const Q& o) const {
       return a * o.b < o.a * b;
   }
   friend ostream& operator<<(ostream& cout, const Q& o) {
       return cout << o.a << "/" << o.b;
   }
};
```

Strongly Connected Components (Luke Videckis): 8940ca

```
struct scc_info {
   int num_sccs; /**< number of SCCs */
   /**
    * scc_id[u] = id of SCC containing node u. It satisfies:
    * - 0 <= scc_id[u] < num_sccs
    * - for each edge u -> v: scc_id[u] >= scc_id[v]
    */
   vector<int> scc_id;
};

scc_info scc(const vector<vector<int>>& adj) {
   int n = adj.size(), timer = 1, num_sccs = 0;
   vector<int> tin(n), scc_id(n, -1), node_stack;
   node_stack.reserve(n);
   auto dfs = [&](auto&& self, int u) -> int {
       int low = tin[u] = timer++;
       node_stack.push_back(u);
       for (auto v : adj[u])
           if (scc_id[v] < 0)
               low = min(low, tin[v] ? tin[v] : self(self, v));
       if (tin[u] == low) {
           while (1) {
               int node = node_stack.back();
               node_stack.pop_back();
               scc_id[node] = num_sccs;
               if (node == u) break;
           }
           num_sccs++;
       }
       return low;
   };
   for (int i = 0; i < n; i++)
       if (!tin[i])
           dfs(dfs, i);
   return {num_sccs, scc_id};
}
```

Ford Fulkerson Max Flow: 78dd3a

```
typedef vector<int> vi;
typedef vector<bool> vb;
struct flowGraph{
   int INF = 1e9;
   vector<vi> cap; vb seen;
   int nodes, so, si;
   flowGraph(int n, int source, int sink){
       nodes = n; so = source; si = sink;
       cap = vector<vi>(n,vi(n,0));
       seen = vb(n);
   }
   void updateEdge(int i, int j, int c){ cap[i][j]+=c; }
   int dfs(int i, int flow, vb& seen){
       if(i == si) return flow;
       seen[i] = 1;
       for(int j = 0; j < nodes; j++){
           if(cap[i][j] && !seen[j]){
               if(int sent = dfs(j,min(flow,cap[i][j]),seen)){
                   cap[i][j] -= sent;
                   cap[j][i] += sent;
                   return sent;
               }
           }
       }
       return 0;
   }
   int maxflow(){
       int mf = 0, sent = -1;
       while(sent != 0){
           seen = vb(nodes);
           sent = dfs(so,INF,seen);
           mf += sent;
       }
       return mf;
   }
};
```

KMP/Prefix Function/Single String matching (Kactl): d4375c

```
#define rep(i,a,b) for(int i = a; i < (b); i++)
#define sz(a) (int)a.size()
vi pi(const string& s) {
    vi p(sz(s));
    rep(i,1,sz(s)) {
       int g = p[i-1];
       while (g && s[i] != s[g]) g = p[g-1];
       p[i] = g + (s[i] == s[g]);
    }
    return p;
}

vi match(const string& s, const string& pat) {
    vi p = pi(pat + '\0' + s), res;
    rep(i,sz(p)-sz(s),sz(p))
       if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
    return res;
}
```

Automaton: 3b8af8

```
void compute_automaton(string s, vector<vector<int>>& aut) {
    s += '#';
    int n = s.size();
    vector<int> pi = prefix_function(s); //kactle kmp; (above)
    aut.assign(n, vector<int>(26));
    for (int i = 0; i < n; i++) {
        for (int c = 0; c < 26; c++) {
            if (i > 0 && 'A' + c != s[i])
                aut[i][c] = aut[pi[i-1]][c];
            else
                aut[i][c] = i + ('A' + c == s[i]);
        }
    }
}

//we have some string and add character 'X' to the end
//what is the prefix of the string that is the valid suffix of the new
//string we created, useful for turning into a graph + matrix expo
```

Hash Concat Kacktl Hashes, a+b, takes/returns pair (Hash,length): c92606

```
pair<H,int> concat(const pair<H,int> &a, const pair<H,int> &b){
    pair<H,int> res;
    res.second = a.second+b.second;
    res.first = (pw[b.second] * a.first) + b.first; //precompute pw arr
    return res;
}
```

Trie: full hash 0dd7e5, no kth: 19d471

```
const int ALPHASIZE = 26; //assumes alphabet is continues
struct node {
   bool isWord = false; //int ct for kth lex
   vector<node*> children;
   node() {
       children = vector<node*>(ALPHASIZE, nullptr);
   } //change -'a' to the first letter of the alphabet.
   void insert(string &s, int idx) { //ct++ at top for kth lex
       if(idx == s.size()) {isWord = true; return;}
       if(!children[s[idx]-'a'])
           children[s[idx]-'a'] = new node();
       children[s[idx]-'a'] -> insert(s, idx+1);
   }

   bool find(string &s, int idx) {
       if(idx == s.size()) return isWord;
       if(!children[s[idx]-'a']) return false;
       return children[s[idx]-'a'] -> find(s, idx+1);
   }
//stop here if don’t need find kth
   void findkth(string &s, int k){
       int sum = 0;
       k-=isWord;
       if(!k) return;
       for(auto i: order){  //order of alphabet is 0,1,2-
           if(children[i] == nullptr) continue;
           if((sum+(children[i]->ct)) >= k){
               s.push_back(i+'a');
               children[i]->findkth(s,k-sum);
               return;
           }
           sum+=(children[i]->ct);
       }
   }
};
```

Aho-corasick DFA: af351d

```
const int K = 26;
const char fir = 'a';
struct Vertex {
    int next[K];
    bool output = false;
    int p = -1;
    char pch;
    int link = -1;
    int go[K];
    int id = -1;
    Vertex(int p=-1, char ch='$') : p(p), pch(ch) {
        fill(begin(next), end(next), -1);
        fill(begin(go), end(go), -1);
    }
};

vector<Vertex> t(1);
void add_string(string const& s, int ind) {
    int v = 0;
    for (char ch : s) {
        int c = ch - fir;
        if (t[v].next[c] == -1) {
            t[v].next[c] = t.size();
            t.emplace_back(v, ch);
        }
        v = t[v].next[c];
    }
    t[v].id = ind;
    t[v].output = true;
}

int go(int v, char ch);
int get_link(int v) {
    if(t[v].link != -1) return t[v].link;
    if (v == 0 || t[v].p == 0) t[v].link = 0;
    else t[v].link = go(get_link(t[v].p), t[v].pch);
    return t[v].link;
}

int go(int v, char ch) {
    int c = ch - fir;
    if (t[v].go[c] != -1) return t[v].go[c];
    if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
    else t[v].go[c] = v == 0 ? 0 : go(get_link(v), ch);
    return t[v].go[c];
}
```

TSP //stupid why do i have this in hackpack

```
double tsp(int mask, int lastNode){
   if(__builtin_popcount(mask) == 1) return graph[0][lastNode];
   //if(mask == 0) return 0;
   if(memo[mask][lastNode] >= 0.0) return memo[mask][lastNode];
   double result = 1e9;
   int oldmask = mask - (1 << lastNode);
   for(int i = 0; i < nodes; i++){
       if(!(oldmask & (1<<i))) continue;
       double temp = tsp(oldmask,i)+graph[i][lastNode];
       result = min(result,temp);
   }
   return memo[mask][lastNode] = result;
}

double minCost = 1e9;
for(int i = 0; i < nodes; i++){
   minCost = min(minCost,tsp((1<<nodes)-1,i)+graph[i][0]);
}
```

Recursive Permutations

```
void permute(vector<int> &ordering, vector<bool> &used, int k){
   if(k == ordering.size()){ //do stuff
   }else{
       for(int i = 0; i < ordering.size(); i++){
           if(used[i]) continue;
           ordering[k] = i;
           used[i] = true;
           permute(ordering,used,k+1);
           used[i] = false;
       }
   }
}
vector<int> ordering(n); vector<bool> used(n); permute(ordering,used,0);
```

Coordinate Compress

```
vals = vector of big numbers
int n = vals.size();
vector<pair<int,int>> info(n);
for(int i = 0; i < n; i++) info[i] = make_pair(vals[i],i);
sort(info.begin(),info.end());
int id = 0;
for(int i = 0; i < n; i++){
   if(i && info[i-1].first != info[i].first) id++;
   vals[info[i].second] = id;
}
```

2D Binary Index Tree (Not mine): 3cc106

```
//one based also maybe make long
struct BIT2D {
      vector<vector<int>> arr;
      int n, m;
      BIT2D(int n, int m): n(n), m(m), arr(n+1, vector<int>(m+1)) {}
      void update(int x, int y, int a) {
            for (; x <= n; x += x&-x)
                  for (int j = y; j <= n; j += j&-j) arr[x][j] += a;
      }
      int query(int x, int y) {
            int s = 0;
            for (; x > 0; x -= x&-x)
                  for (int j = y; j > 0; j -= j&-j ) s+= arr[x][j];
            return s;
      }

    int query(int x1, int y1, int x2, int y2){
        return query(x2, y2) - query(x1-1, y2) - query(x2, y1-1) +
query(x1-1, y1-1);
    }
};
```

Palindrome Tree/eertree: ce5455

Pick's Theorem

$A_{poly} = I + \frac{B}{2} - 1$

A_{poly} - Area of the polygon (must be on lattice points)

I - # of lattice points strictly inside the polygon

B - # of boundary points

Geometry

Domain and Range

Function	Domain	Range
$y = \sin^{-1} x$	$-1 \le x \le 1$	$-\frac{\pi}{2} \le y \le \frac{\pi}{2}$
$y = \cos^{-1} x$	$-1 \le x \le 1$	$0 \le y \le \pi$
$y = \tan^{-1} x$	$-\infty < x < \infty$	$-\frac{\pi}{2} < y < \frac{\pi}{2}$

$x = r\cos\theta$

$y = r\sin\theta$

$r = \sqrt{x^2 + y^2}$

$\theta = \tan^{-1}(\frac{y}{x})$

Law of Sines

$\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c}$

Law of Cosines

$a^2 = b^2 + c^2 - 2bc\cos\alpha$

$b^2 = a^2 + c^2 - 2ac\cos\beta$

$c^2 = a^2 + b^2 - 2ab\cos\gamma$

Vector Projections / Euler Characteristic

$proj_v u = \left(\frac{u \bullet v}{\lvert v \rvert^2}\right) v$ Projection of u onto v	V - E + F = 2

Bitwise stuff

Set Bit	Mask = Mask |(1<<I)
Clear Bit	Mask = Mask & ~(1<<I)
Toggle Bit	Mask = Mask^(1<<I)
Convert trailing 0's to 1's Example: 01101000 -> 01101111	Mask = Mask | (Mask-1)
Least Significant 1 Bit	Mask & -Mask
Change individual byte of 4 byte integer	int x1 = 0; ((uint8_t*) &x1)[0] = 1

	((uint8_t*) &x1)[1] = 1 **((uint8_t*) &x1)[2] = 1** **((uint8_t*) &x1)[3] = 1**
Iterate over Submasks	**for (int sb=mask; sb; sb=(sb-1)&mask)** **3^N, DOES NOT PROCESS SUBMASK 0**
Count Ones **Count leading Zeros** **Count trailing Zeros** **Most Significant Bit Index (0)**	**__builtin_popcount(n)** **__builtin_clz(n)** **__builtin_ctz(n** **__lg(n)**

Dsu Bipartite: 132f76

```
struct dsu{
    vector<int> sz; vector<pair<int,int>> parent;
    bool isBipartite = true;
    dsu(int n){
        isBipartite = true; sz = vector<int>(n,1);
        parent = vector<pair<int,int>>(n);
        for(int i = 0; i < n; i++) parent[i] = {i,0};
    }
    pair<int,int> find(int a){
        if(a != parent[a].first){
            int parity = parent[a].second;
            parent[a] = find(parent[a].first);
            parent[a].second ^= parity;
        }
        return parent[a];
    }
    bool join(int aIn, int bIn){
        auto[a,parityA] = find(aIn); auto[b,parityB] = find(bIn);
        if(a==b) return (parityA!=parityB);
        else{
            if(sz[a]<sz[b]) swap(a,b); sz[a]+=sz[b];
            parent[b] = {a,parityA^parityB^1};
        }
        return isBipartite;
    }
};
```

Dsu Rollback: 329f3e

```
struct dsuR{
    vector<int> sz,par; int comps;
    vector<tuple<int,int,int,int>> changes;
    dsuR(int n){
        sz = par = vector<int>(n,1); comps = n;
        iota(par.begin(),par.end(),0);
    }
    int find(int u){ if(par[u] != u) u = find(par[u]); return u; }
    int time(){ return changes.size();} dsuR(){}
    bool join(int u, int v){
```

```
        u = find(u); v = find(v); if(u==v) return false;
        if(sz[u] < sz[v]) swap(u,v); //u is now big dude
        changes.emplace_back(u,sz[u],v,par[v]); //store previous
        sz[u]+=sz[v]; par[v] = u; comps--; return true;
    }
    void rollback(int t){
        while((int)changes.size() != t){
            auto[a,oldsza,b,oldparb] = changes.back();
            changes.pop_back();
            sz[a] = oldsza; par[b] = oldparb; comps++;
        }
    }
};
```

Dynamic Connectivity: 66a463

```
struct node{
    int lo,hi,mid;
    vector<pair<int,int>> edges;
    node *left; node *right;
    node(int l, int r){
        lo = l; hi = r; mid = (lo+hi)/2;
        if(lo==hi) return;
        left = new node(l,mid);
        right = new node(mid+1,r);
    }

    void addInterval(int l, int r, const pair<int,int> &edge){
        if(l>hi || r < lo) return; //nocover
        if(l<=lo&&hi<=r){ edges.emplace_back(edge); return; }
        left->addInterval(l,r,edge);
        right->addInterval(l,r,edge);
    }

    void dfs(dsuR &ds,vector<int> &ans){
        int t = ds.time();
        for(auto[u,v]: edges) ds.join(u,v);
        if(lo==hi){
            ans[lo] = ds.comps; //Answer Query
        }
        else { left->dfs(ds,ans); right->dfs(ds,ans); }
        ds.rollback(t);
        return;
    }
};

vector<int> dc(vector<tuple<int,int,int>> &queries, int n){
    vector<int> res(queries.size());
    dsuR ds = dsuR(n);
    node tree = node(0,queries.size()-1);
    int ti = 0;
    map<pair<int,int>, int> qTimes;
    for(auto [qu, u, v]: queries){
```

```
        if(u<v) swap(u,v);
        if(qu == 1) qTimes[{u,v}] = ti;
        if(qu == 2) {
            tree.addInterval(qTimes[{u,v}],ti-1,{u,v});
            qTimes.erase({u,v});
        }
        ti++;
    }
    for(auto[edge,tAdded]: qTimes) tree.addInterval(tAdded,ti,edge);
    tree.dfs(ds,res);
    return res;
}
```

Largest and Smallest Triangle (n^2, n^2logn)

```
ll smallestTriangle(vector<P> pts){
    sort(pts.begin(),pts.end());
    if(pts.size() < 3) return 1e18;
    typedef tuple<P,int,int> vinfo;
    vector<vinfo> vecs;
    int n = pts.size(); ll smallest = 1e18;
    vector<int> ind(n); iota(ind.begin(),ind.end(),0);
    for(int i = 0; i < n; i++)
        for(int j = i+1; j < n; j++)
            vecs.emplace_back(pts[j]-pts[i],i,j);

    sort(vecs.begin(),vecs.end(), [&](vinfo av, vinfo bv)->bool {
        P a = get<0>(av); P b = get<0>(bv);
        bool sideA = a.x > 0 || (a.x == 0 && a.y > 0);
        bool sideB = b.x > 0 || (b.x == 0 && b.y > 0);
        if(sideA != sideB) return sideA;
        return a.cross(b) > 0;
    });

    for(auto[p,aorig,borig]: vecs){
        int a = ind[aorig]; int b = ind[borig];
        int le = min(a,b)-1; int ri = max(a,b)+1;
        if(le >= 0) smallest =
min(smallest,abs(pts[le].cross(pts[a],pts[b])));
        if(ri != n) smallest =
min(smallest,abs(pts[ri].cross(pts[a],pts[b])));
        swap(pts[a],pts[b]); swap(ind[aorig],ind[borig]);
    }

    return smallest;
}

ll largestTriangle(const vector<P> &pts){
    vector<P> hull = convexHull(pts);
    int hs = hull.size(); if(hs < 3) return 0;
    auto area = [&](int a, int b, int c){ return
```

```
hull[a].cross(hull[b],hull[c]); };
    auto next = [&](int x) {return (x+1 == hs) ? 0 : x+1; };
    int a = 0; int b = next(a); int c = next(b);
    ll largest = hull[a].cross(hull[b],hull[c]);
    while(true){
        while(a!=c){
            while(area(a,b,next(c)) >= area(a,b,c)) c = next(c);
            if(area(a,b,c) >= largest) largest = area(a,b,c);
            b = next(b);
        }
        a = next(a); if(!a) break;
        b = next(a); c = next(b); // readline carefully
    }
    return largest;
}
```

Tomasz Trick (Keyboard queries, Listing Passwords) (491a4a)

```
const int LOG = 20;
vector<dsu> dsus;
void join2(int u, int v, int pow){
    if(!dsus[pow].join(u,v)) return;
    if(pow == 0) return;
    int half = 1<<(pow-1);
    join2(u,v,pow-1);
    join2(u+half,v+half,pow-1);
};

void joinranges(int u, int v, int len){
    int pow2 = (1 << __lg(len));
    int overlap = len-pow2;
    join2(u,v,__lg(len));
    join2(u+overlap,v+overlap,__lg(len));
}

bool check2(int u, int v, int pow){
    if(dsus[pow].find(u) == dsus[pow].find(v)) return true;
    if(pow == 0) return false;
    int half = 1<<(pow-1);
    if(check2(u,v,pow-1) && check2(u+half,v+half,pow-1)){
        dsus[pow].join(u,v); return true;
    }
    return false;
}

bool check(int u, int v, int len){
    int pow2 = (1<<__lg(len)); int overlap = len-pow2;
    return check2(u,v,__lg(len)) &&
check2(u+overlap,v+overlap,__lg(len));
}

//Dealing with palindromes
//for(int i = 0; i < n; i++) dsus[0].join(i,2*n-i-1);
```

```
//joinranges(u,2*n-v-1,len);
//change problem from isPalindrome to, these two substrings are equal
//STRING|REVERSEDSTRING
```

Prioritized Matching, (Lex minimal)

```
vi minimizeMatching(vvi& g, vi& btoa) {
    int n = sz(g), m = sz(btoa);
    vi matching(n), blocked(m), seen(m);
    rep(i, 0, m) if(btoa[i] != -1) matching[btoa[i]] = i;
    auto dfs = [&](int b, int tar, auto &dfs) -> bool {
        int cur = btoa[b];
        if(cur == -1) {
            btoa[matching[tar]] = -1;
            return true;
        }
        if(cur == tar || cur == -1) return true;
        seen[b] = true;
        for(int nxt: g[cur]) if(!seen[nxt] && dfs(nxt, tar, dfs)) {
            matching[cur] = nxt, btoa[nxt] = cur;
            return true;
        }
        return false;
    };
    rep(i, 0, n) for(int j: g[i]) if(!blocked[j]) {
        seen = blocked;
        if(dfs(j, i, dfs)) {
            matching[i] = j, btoa[j] = i, blocked[j] = true;
            break;
        }
    }
    return matching;
}
//pass in btoa array returned from dfs matching
//READ notes from dfs matching to explain btoa array
//this prioritizes giving first left guy to first element
//in its neighbor array then assigning it to second in its neighbor
//etc, then send left guy's prioritization
//sort neighbor arrays for lex min
//or sort backwards for lex max
```

Li Chao Tree, finds max point

```
struct func {
    ll E, R;
    func(ll E = -1, ll R = -1): E(E), R(R) {}
    ll operator()(ll x) {
        if(E == -1) return -1e9;
        return (R - P[x]) * (E - S[x] + 1);
    }
};

struct node {
    int lo, md, hi;
    func f;
    node *left, *right;
    node(int lo, int hi): lo(lo), hi(hi), md((lo + hi)>>1) {
        if(lo == hi) return;
        left = new node(lo, md);
        right = new node(md+1, hi);
    }
    void update(func e) {
        if(e(md) > f(md)) swap(e, f);
        if(lo == hi) return;

        if(e(lo) < f(lo) && e(hi) < f(hi)) return;

        if(e(lo) > f(lo)) left->update(e);
        else right->update(e);
    }
    void rangeUpdate(func e, int L, int R) {
        if(R < lo || hi < L) return;
        if(L <= lo && hi <= R) return update(e);
        left->rangeUpdate(e, L, R);
        right->rangeUpdate(e, L, R);
    }
    ll query(int x) {
        if(lo == hi) return f(x);
        if(x <= md) return max(f(x), left->query(x));
        return max(f(x), right->query(x));
    }
};

//switch signs and max for minimal position
```

Centroid Decomp

```
//vector<int> blocked,subSize; //size of tree + extra if you want
int sSize(int u, int p = -1) {
      subSize[u] = 1;
      for (int v : graph[u]) {
            if (v == p || blocked[v]) continue;
            subSize[u] += sSize(v,u);
      }
      return subSize[u];
}

int get_centroid(int u, int n, int p = -1) {
      for (int v : graph[u]) {
            if (v == p || blocked[v])  continue;
            if (subSize[v] * 2 > n) return get_centroid(v, n, u);
      }
      return u;
}

ll centroid_decomp(int u = 0) {
    int centroid = get_centroid(u, sSize(u));
    ll ans = solve(centroid);
    blocked[centroid] = true;
    for (int v : graph[centroid]) {
      if (blocked[v]) continue;
      ans+=centroid_decomp(v);
    }
    return ans;
}
```

SAM (Jacob's): 1914a9

```
// len is the longest-length substring ending here
// pos is the first index in the string matching here
// term is whether this node is a terminal (aka a suffix)
struct st { int len, pos, term; st *link; map<char, st*> next; };
st *suffixAutomaton(string &str) {
    st *last = new st(), *root = last;
    for(auto c : str) {
        st *p = last, *cur = last = new st{last->len + 1, last->len};
        while(p && !p->next.count(c))
            p->next[c] = cur, p = p->link;
        if (!p) cur->link = root;
        else {
            st *q = p->next[c];
            if (p->len + 1 == q->len) cur->link = q;
            else {
                st *clone = new st{p->len+1, q->pos, 0, q->link,
q->next};
                for (; p && p->next[c] == q; p = p->link)
                    p->next[c] = clone;
                q->link = cur->link = clone;
            }
        }
    }
    while(last) last->term = 1, last = last->link;
    return root;
}
```

Treap:

```
struct node {
    char val;
    int prior, sz = 1;
    node *left = nullptr, *right = nullptr;
    node(char val = 0): val(val), prior(rand()) {}
};

int getSz(node *cur) { return cur ? cur->sz : 0; }

void recalc(node *cur) { cur->sz = getSz(cur->left) + getSz(cur->right)
+ 1; }

pair<node*, node*> split(node *cur, int v) {
    if(!cur) return {nullptr, nullptr};
    node *left, *right;
    if(getSz(cur->left) >= v) {
        right = cur;
        auto [L, R] = split(cur->left, v);
        left = L, right->left = R;
        recalc(right);
    }
    else {
        left = cur;
        auto [L, R] = split(cur->right, v - getSz(cur->left) - 1);
        left->right = L, right = R;
        recalc(left);
    }
    return {left, right};
}

node* merge(node *t1, node *t2) {
    if(!t1 || !t2) return t1 ? t1 : t2;
    node *res;
    if(t1->prior > t2->prior) {
        res = t1;
        res->right = merge(t1->right, t2);
    }
    else {
        res = t2;
        res->left = merge(t1, t2->left);
    }
    recalc(res);
    return res;
}
```

Morbin Time: (example -> n integers, count pairs of integers are coprime)

```
vector<int> freq(maxv);
for(int i = 0; i < n; i++){
    int v; cin >> v; freq[v]++;
}
//morbin time (mobius function)
vector<int> mu(maxv); mu[1] = 1;
for(int i = 1; i < mu.size(); i++)
    for(int j = 2*i; j < mu.size(); j+=i)
        mu[j]-=mu[i];
//no longer morbin time
ll ans = 0;
for(int d = 1; d<maxv; d++){
    ll sum = 0;
    for(int j = 0; j < maxv; j+=d) sum+=freq[j];
    ans+=(mu[d]*choose2(sum));
}
```

For any positive integer n,

$$\mu(n) = \begin{cases} 1 & \text{if } n = 1, \\ 0 & \text{if } a^2 \mid n \text{ for some } a > 1 \text{ (i.e., } n \text{ has a squared prime factor)}, \\ (-1)^k & \text{if } n \text{ is the product of } k \text{ distinct primes.} \end{cases} \square$$

HLD

```
vector<int> parent, depth, heavy, head, pos;
int cur_pos;
int dfs(int v, vector<vector<int>> const& adj) {
    int size = 1;
    int max_c_size = 0;
    for (int c : adj[v]) {
        if (c != parent[v]) {
            parent[c] = v, depth[c] = depth[v] + 1;
            int c_size = dfs(c, adj);
            size += c_size;
            if (c_size > max_c_size)
                max_c_size = c_size, heavy[v] = c;
        }
    }
    return size;
}

void decompose(int v, int h, vector<vector<int>> const& adj) {
    head[v] = h, pos[v] = cur_pos++;
    if (heavy[v] != -1)
        decompose(heavy[v], h, adj);
    for (int c : adj[v]) {
        if (c != parent[v] && c != heavy[v])
            decompose(c, c, adj);
    }
}

void init(vector<vector<int>> const& adj) {
    int n = adj.size();
    parent = depth = head = pos = vector<int>(n);
    heavy = vector<int>(n, -1);
    cur_pos = 0;
    dfs(0, adj);
    decompose(0, 0, adj);
}
//Declare segtree st
int query(int a, int b) {
    int res = 0;
    for (; head[a] != head[b]; b = parent[head[b]]) {
        if (depth[head[a]] > depth[head[b]])
            swap(a, b);
        int cur_heavy_path_max = st.query(pos[head[b]], pos[b]);
        res = max(res, cur_heavy_path_max);
    }
    if (depth[a] > depth[b])
        swap(a, b);
    int last_heavy_path_max = st.query(pos[a], pos[b]);
    res = max(res, last_heavy_path_max);
    return res;
}
```

Usage:

```
int main(){
    cin.tie(0)->sync_with_stdio(0);
    int n,q; cin >> n >> q;
    vector<int> vals(n); for(auto &x: vals) cin >> x;
    vector<vector<int>> graph(n);
    for(int i = 0; i < n-1; i++){
        int u,v; cin >> u >> v; u--; v--;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
    st = SegTree(n);
    init(graph);
    for(int i = 0; i < n; i++) st.update(pos[i],vals[i]);
    for(int i = 0; i < n; i++){
        int qt; cin >> qt;
        if(qt==1){
            int v,w; cin >> v >> w; v--;
            st.update(pos[v],w);
        }else{
            int u,v; cin >> u >> v; u--; v--;
            cout << query(u,v) << " ";
        }
    }

    return 0;
}
```